

scikit-learn, on one card

Every model wears the same coat: make it, `.fit(train)`, `.predict()`. Split first, scale inside a Pipeline, and judge it by the metric the mistake actually costs. Module 2, Sessions 7 to 13.

girishmkulkarni.github.io/applied-maths-in-industry-site

Dr. Girish Kulkarni

Every model, the same few lines

Split, fit, predict

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```

```
model.fit(X_train, y_train)      # learn from the training
                                # half
pred = model.predict(X_test)     # guess on the held-out half
model.score(X_test, y_test)     # a quick built-in score
→ Every model in this module -- line, tree, forest, k-means -- wears this same coat:
make it, .fit(), .predict(). Learn it once.
```

Lock the test set away first

```
# split BEFORE you scale, fill gaps, or fit anything
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

! Anything the model learns must come from TRAIN only. Fit a scaler, a filler, or an
encoder on the whole data and the test score is a flattering lie -- that is data leakage,
the No. 1 way beginners fool themselves.
→ random_state fixes the split so the run is repeatable.
```

Not just yes/no -- how sure

```
model.predict(X_test)           # the class: 0 or 1
proba = model.predict_proba(X_test) # two columns of P
p_yes = proba[:, 1]             # P(class 1), e.g. 0.83
→ The probability lets you RANK cases and set your own threshold -- vital when one
class is rare (S09).
```

Predict a number -- regression (S07-S08)

The straight-line model

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
model.coef_      # the weight on each feature
model.intercept_ # where the line meets zero
model.predict([[1200, 3]]) # a new flat -> a price
→ Same least-squares line as np.linalg.lstsq in S02, now behind a tidy .fit / .predict.
```

Steadier weights: Ridge & Lasso

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
```

```
Ridge(alpha=10.0)      # shrink every weight toward 0
Lasso(alpha=0.1)       # shrink, and zero the useless ones
ElasticNet(alpha=0.1, l1_ratio=0.5) # a bit of both
→ alpha is the one knob: bigger = more shrinkage. It trades a snugger fit for steadier
predictions on flats the model never saw. Choose it by cross-validation, never by hand.
```

Did the number land close?

```
from sklearn.metrics import (mean_absolute_error,
                             mean_squared_error, r2_score)

mean_absolute_error avg miss, in the real unit (Rs)
mean_squared_error  punishes big misses harder
mse ** 0.5          RMSE -- back in the real unit
r2_score            1 = perfect, 0 = no better than the mean
→ Report MAE to a human (it is in rupees); watch R2 to compare models. A negative R2
means worse than guessing the average.
```

Predict yes or no -- classification (S09)

Logistic regression, and how sure

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
model.predict(X_test)           # 0 / 1
model.predict_proba(X_test)[:, 1] # P(class 1)
→ "Regression" in the name, but it answers a yes/no question -- it fits a probability,
then cuts at 0.5.
```

The four kinds of right and wrong

```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(y_test, pred)
# [[TN FP] rows = truth, cols = guess
# [FN TP]] FN = a real 'yes' we missed
→ Every classifier score is built from these four counts. In fraud or default, the costly
box is FN -- the case you let through.
```

Scoring a classifier

```
from sklearn.metrics import (accuracy_score,
                             precision_score, recall_score, f1_score, roc_auc_score)

accuracy_score share correct (misleads when rare)
precision_score of the alarms, how many were real
recall_score    of the real yeses, how many we caught
f1_score        balance of precision and recall
roc_auc_score   ranking quality; feed it the PROBA
! roc_auc_score wants probabilities, not 0/1 labels: roc_auc_score(y_test, p_yes).
```

When accuracy lies

```
LogisticRegression(class_weight="balanced")
RandomForestClassifier(class_weight="balanced")

! If 'yes' is 2% of the data, a model that always says 'no' scores 98% accuracy and
catches zero fraud. Judge it by recall and PR-AUC, not accuracy.
→ class_weight="balanced" makes a mistake on the rare class cost more, so the model
stops ignoring it.
```

Flowcharts that learn -- trees & ensembles (S10)

One decision tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
tree = DecisionTreeClassifier(max_depth=3, random_state=0)
tree.fit(X_train, y_train)
plot_tree(tree, feature_names=cols, filled=True)
```

! Leave `max_depth` off and the tree grows until it memorises the training rows -- 100% train, poor test. That is overfitting; cap the depth.

Many trees beat one

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier # pip install xgboost
```

```
RandomForestClassifier(n_estimators=200, random_state=0)
XGBClassifier(n_estimators=200, learning_rate=0.1,
              max_depth=3, random_state=0)
```

→ Forest: grow many trees on random samples and let them VOTE. XGBoost: grow trees in sequence, each fixing the last one's mistakes. Both are the workhorses of tabular ML; Colab may need pip install xgboost.

Which inputs mattered

```
importances = forest.feature_importances_
order = importances.argsort()[::-1]
for i in order:
    print(cols[i], round(importances[i], 3))
→ A first read on what the model leaned on -- the start of the bias audit in S10's third notebook.
```

sklearn's own booster

BEYOND CLASS

```
from sklearn.ensemble import GradientBoostingClassifier
GradientBoostingClassifier(n_estimators=200,
                           learning_rate=0.1, max_depth=3)
```

→ The same boosting idea, built into scikit-learn with no extra install. The course reaches for XGBoost instead, but you will meet this one in the wild.

Get the data model-ready (S08, S12)

Put every column on one scale

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
scaler.fit(X_train)                # learn mean & sd on TRAIN
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test) # reuse the same numbers
```

! Never .fit the scaler on the test set. Fit on train, then transform both. Distance and regularised models (k-means, Ridge, logistic) are useless without it.

Text columns -> numbers

```
import pandas as pd
pd.get_dummies(df, columns=["city"]) # one 0/1 column each
```

```
from sklearn.preprocessing import OneHotEncoder
OneHotEncoder(handle_unknown="ignore")
```

→ Models eat numbers, not the word 'Pune'. get_dummies is the quick way; OneHotEncoder is the one that slots into a Pipeline and remembers the training categories.

One object, no leakage: Pipeline

```
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0)),
])
pipe.fit(X_train, y_train) # scales, THEN fits, as one
```

→ The scaler is re-fitted fresh inside every cross-validation fold, so no test information can leak in. Bundle prep + model, always.

New features from old

BEYOND CLASS

```
from sklearn.preprocessing import PolynomialFeatures
PolynomialFeatures(degree=2) # x -> x, x^2, ...
```

```
df["dist"] = np.sqrt(df.x**2 + df.y**2) # a made feature
```

→ The right feature can turn an impossible split into an easy one -- two rings that no line separates come apart on distance-from-centre (S12).

Let the data group itself (S11)

Find the groups: k-means

```
from sklearn.cluster import KMeans
```

```
km = KMeans(n_clusters=3, random_state=0, n_init=10)
labels = km.fit_predict(X_scaled) # a group id per row
km.inertia_ # tightness (lower better)
```

→ No answer key -- the data sorts itself into clumps. Scale first, or the column with the biggest numbers decides everything. Try a few k and look for the elbow in inertia (or use silhouette_score).

Squash many columns to two: PCA

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
XY = pca.fit_transform(X_scaled) # n columns -> 2
pca.explained_variance_ratio_ # spread kept per axis
```

→ Turns a 5- or 50-column table into a 2-D picture you can actually plot, keeping as much spread as it can. Cluster to find groups, PCA to see them.

Groups of any shape: DBSCAN

BEYOND CLASS

```
from sklearn.cluster import DBSCAN
DBSCAN(eps=0.3, min_samples=5) # finds dense blobs
```

→ k-means always draws round groups and must be told how many; DBSCAN finds however many dense regions there are, any shape, and tags the outliers as -1.

Choose the model honestly (S08)

Don't trust one split

```
from sklearn.model_selection import cross_val_score,
GridSearchCV
```

```
cross_val_score(model, X_train, y_train, cv=5) # 5 scores
```

```
grid = GridSearchCV(pipe, {"model__alpha": [0.1, 1, 10]},
                    cv=5, scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)
grid.best_params_
```

→ Cross-validation splits the TRAIN set into folds and averages, so one lucky split can't fool you. GridSearchCV wraps it to pick the best knob (alpha, depth) for you.

Which metric, when

regression	MAE for a human, R2 to compare
balanced classes	accuracy is fine
rare 'yes' (fraud)	recall + PR-AUC, NOT accuracy
ranking cases	roc_auc_score on the probabilities
clustering	silhouette_score (no labels to check)

→ Pick the score before you train, from what a mistake actually costs -- not the one that looks highest.

Mistakes everyone makes once

! Scaling or filling gaps BEFORE the split leaks the test set into training. Split first, or use a Pipeline.

! Tuning knobs against the test set turns it into a second training set. Tune with cross-validation; touch the test set once, at the very end.

! Judging a rare-class model by accuracy. A 98% score can mean it caught nothing. Look at recall.

! Forgetting to scale before k-means, PCA, or a distance model -- the biggest-numbered column silently wins.

! Leaving out random_state, so every run gives a different answer and nothing is reproducible.

! fit_transform on train, but fit_transform again on test -- use transform on test, so it reuses the train fit.